

“Learning Platform with AI Doubt Solver”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

KOUSHIK S	2520090113
A. SAI CHARAN TEJA	2520080044
CH. GANESH	2520090112

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Learning Platform with AI Doubt Solver”** is a record of bonafide work of **KOUSHIK S – 2520090113, A. SAI CHARAN TEJA – 2520080044, CH. GANESH – 2520090112** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 5, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

KOUSHIK S – 2520090113

A. SAI CHARAN TEJA – 2520080044

CH. GANESH – 2520090112

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Learning Platform with AI Doubt Solver”** is a bonafide work done and submitted by **KOUSHIK S – 2520090113, A. SAI CHARAN TEJA – 2520080044, CH. GANESH – 2520090112** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 5, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator, Mr. Sateesh Kumar, and our guide, Divya Priya Degala, Department of Computer Science and Information Technology, whose tremendous support, encouragement and guidance cannot be expressed in words. Her direction helped us organize the database, backend and user-interface work into a coherent academic project.

We express our gratitude to Dr. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We also acknowledge the official documentation of React, Vite, FastAPI, SQLAlchemy, PostgreSQL, pgvector, MongoDB, Axios, pytest, Docker and Vercel, which supported the implementation and troubleshooting of the project.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

KOUSHIK S – 2520090113

A. SAI CHARAN TEJA – 2520080044

CH. GANESH – 2520090112

Abstract

The Learning Platform with AI Doubt Solver is a web-based educational platform designed to combine structured course management with context-aware academic doubt resolution. Students often become blocked when they meet a technical concept and cannot obtain timely clarification, while traditional learning portals keep course content, enrollment information and instructor communication separate. The proposed system provides students with a course catalog, enrollment facilities, course modules and learning content, while also maintaining a historical ledger of doubts and their responses. An instructor-oriented view is provided for monitoring student inquiries and course-level doubt activity.

The application uses a React and Vite frontend and a layered FastAPI backend. PostgreSQL stores the normalized relational data — users, courses, enrollments, modules, course content and doubts — and, through the pgvector extension, 1536-dimensional content embeddings on which cosine-distance similarity search is performed. MongoDB is included for flexible document-oriented entities such as course assets and doubt attachments. The database layer demonstrates advanced SQL through a trigger that marks a doubt as answered when a response is inserted, an idempotent stored procedure for enrollment, and an analytics view for course-level doubt counts.

The AI doubt-solving workflow follows a retrieval-oriented design. When a student submits a question, the backend stores the doubt, creates a deterministic 1536-dimensional embedding, retrieves the top relevant content chunks using pgvector, generates an academic-style answer from the project knowledge base and the retrieved course material, stores the response and its source references, and commits the operation as a single transaction. The embedding and answer-generation functions are intentionally isolated so that a production embedding model or external LLM can be substituted later. Authentication uses bcrypt password hashing and JWT access tokens, with role-based access control protecting instructor-oriented analytics, and automated pytest tests cover authentication, course APIs, enrollment behavior and end-to-end doubt submission.

Keywords: Learning Platform, AI Doubt Solver, FastAPI, React, PostgreSQL, MongoDB, pgvector, RAG, JWT, REST API, SQL, Vector Search, Database Engineering.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	8
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	10
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	11
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	12
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	14
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	16
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	18
8	Testing	8.1 Postman and Swagger API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	19
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	21
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	22

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	23
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	24
13	References	Documentation, tools, and resources used	25
14	Appendices	Sample Code Snippets Installation / Setup Instructions Default Login Credentials API Reference Summary Screenshots and Project Poster	26

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Online learning systems make course material accessible, but students can still become blocked when they encounter a technical concept and cannot obtain timely clarification. A traditional learning portal usually separates course content, enrollment information and instructor communication. The proposed platform combines these areas with a dedicated doubt-solving workflow.

The project is designed around the idea of a course-aware academic assistant. A doubt is not treated as an isolated chat message; it is stored against a student, a course and an optional module. This context is used by the retrieval pipeline to locate relevant course material and return an answer with source references. The platform therefore connects database engineering with an AI-oriented retrieval workflow.

1.2 Problem Statement

Students need a centralized way to discover courses, access learning modules and obtain fast clarification for technical questions. Without context-aware retrieval, a generic answer may be unrelated to the syllabus. Without persistent storage, students and instructors lose the history of questions and responses. The system must therefore provide structured academic data management together with a retrieval-based doubt-solving mechanism.

1.3 Scope of the Project

The system covers user registration and login with student and instructor roles; a course catalog with course details, modules and learning-content metadata; course enrollment with duplicate-enrollment protection; and student doubt submission with course and module context. It performs vector-based retrieval of relevant course-content chunks using pgvector, generates an AI-style academic answer and stores its source citations, and maintains a doubt history for students and a doubt inbox for instructors. Course-level doubt analytics are provided through a PostgreSQL view, and deployment-ready frontend and container/cloud configuration is included. A cloud-hosted LLM integration and an independently deployed Spring Boot service are outside the scope of the submitted implementation.

1.4 Objectives

- Develop a responsive learning platform for students and instructors.
- Provide a real-time doubt-submission workflow with course-aware context.
- Use PostgreSQL for strongly structured academic and transactional data.
- Use MongoDB for flexible document-style assets and attachments.
- Use pgvector to support semantic retrieval from course-content embeddings.
- Demonstrate advanced SQL through a trigger, a stored procedure and an analytics view.
- Implement JWT authentication, bcrypt password hashing and role-based access control.
- Build the backend using a maintainable routers → services → repositories architecture.

- Provide automated tests for authentication, courses and the doubt-solving workflow.

1.5 Importance of the Application

The application brings learning content and doubt resolution into a single platform. From a database-engineering perspective, it demonstrates why different persistence models may be selected for different workloads: relational records remain normalized and constrained, vector data supports semantic retrieval, and MongoDB provides a flexible structure for media metadata.

The project is also useful as an academic demonstration of how a modern backend can combine authentication, REST APIs, transactions, SQL objects and retrieval-oriented AI components. The architecture isolates the embedding and answer-generation functions so that the current deterministic implementation can later be replaced with production AI services without changing the surrounding API contract.

1.6 Target Users / Audience

The system serves two main roles. Students use it to register and log in, browse the catalog, enroll in courses, view modules, ask doubts and review their doubt history. Instructors use it to review doubts raised across their assigned courses and to inspect course-level doubt analytics. An administrator / system role is supported by the database role model and security layer for future administrative expansion.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The platform is lightweight enough for normal student-development hardware. A modern dual-core or better processor, 8 GB of RAM (recommended for running the database and development services simultaneously) and at least 5 GB of free disk space are sufficient for a local development environment. Docker-based full-stack execution benefits from additional RAM when PostgreSQL, MongoDB and the application containers run together.

2.2 Software Requirements

- Operating System: Windows, Linux or macOS (WSL can be used for Linux-oriented workflows)
- Frontend runtime: Node.js and npm; React 19 with Vite 8 in the submitted package
- Backend runtime: Python 3.11 or later; FastAPI with Uvicorn
- Relational database: PostgreSQL 15+ with the pgvector extension (the repository deployment uses the PostgreSQL 16 pgvector image)
- Document database: MongoDB 6+ (Motor / Mongoose schemas are provided)
- Testing: pytest, pytest-asyncio, pytest-cov and httpx
- Development: VS Code or another Python/JavaScript IDE; Git recommended
- A current web browser (Chrome, Edge or Firefox)

2.3 Development Tools and Frameworks

React with Vite was used for the client application, and Axios for HTTP communication and JWT header attachment. The backend is built with FastAPI, SQLAlchemy and Pydantic. PostgreSQL with pgvector holds the relational and vector data, while MongoDB with Motor/Mongoose holds flexible document collections. Postman is listed in the project poster for API testing, and pytest is implemented in the backend repository. Docker Compose, Dockerfiles, Vercel and Render configuration are included for the deployment options.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a React single-page application built with Vite. React Router provides protected routes for the dashboard, catalog, course details, doubt solver, doubt history and instructor view. Axios centralizes API calls and attaches the stored JWT as a Bearer token through an interceptor. The interface uses a deliberately academic “catalog and ledger” visual style rather than a generic chatbot layout.

3.2 Backend

The real backend in the submitted repository is FastAPI, served by Uvicorn. The code is layered into routers, services, repositories, schemas, database modules and security utilities. SQLAlchemy provides asynchronous PostgreSQL access, while Motor provides asynchronous MongoDB connectivity.

3.3 Database

PostgreSQL stores the core academic model: users, courses, enrollments, modules, course content, doubts, responses, response sources and content embeddings. Foreign keys and unique constraints protect relational integrity, and the schema defines an enum-based role/status model and indexes on frequently queried columns.

pgvector adds a VECTOR(1536) column for content embeddings. The similarity repository orders content using the $\leq$ cosine-distance operator and returns the top three matching chunks.

MongoDB is modeled for flexible course assets and doubt attachments. The Mongoose schema supports media types, tags and a flexible metadata object, and the FastAPI backend initializes a Motor client and exposes collection accessors. Direct CRUD routes for these collections are not currently part of the API surface, so they should be regarded as an integrated persistence layer prepared for asset and attachment expansion.

3.4 Version Control

Git is recommended for version control. The submitted project is organized as a single repository containing the backend, frontend, database (db) and deployment directories.

3.5 Authentication and Security Libraries

Passwords are hashed with bcrypt before storage. JSON Web Tokens (JWT) carry the authenticated user identity and role, while FastAPI dependencies enforce authentication and instructor/admin role checks. Pydantic schemas validate the structure of authentication, enrollment and doubt requests.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system separates the presentation layer from the application services and the persistence layer. Figure 4.1 shows the architecture supplied in the project poster: the user interface, an API gateway/service layer, the FastAPI, Spring Boot and Node/Express service concepts, PostgreSQL, pgvector and MongoDB, with Docker as the containerization boundary.

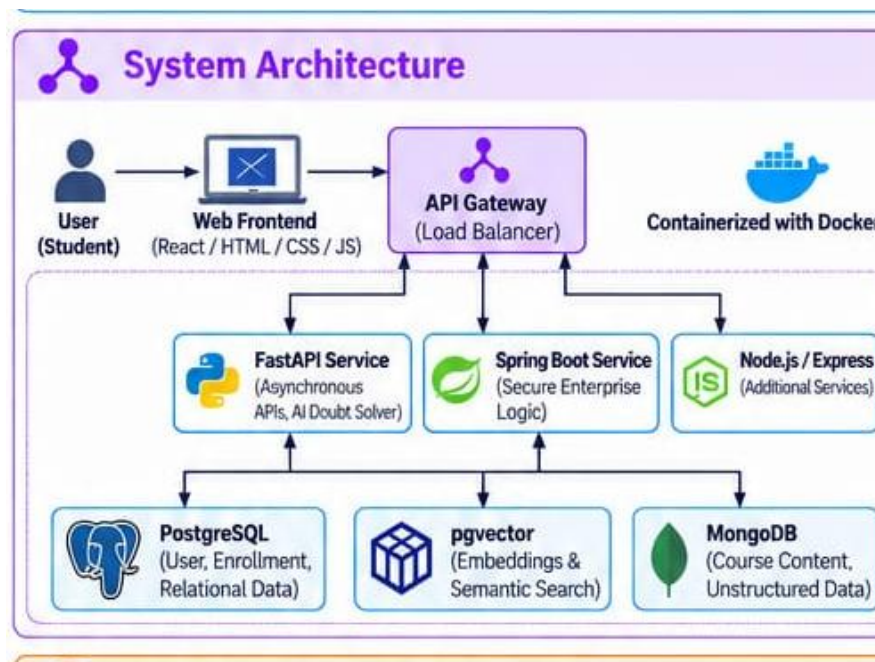


Figure 4.1: System architecture of the Learning Platform with AI Doubt Solver

For the submitted implementation, the operational backend that is actually present in the source repository is the FastAPI service. The Spring Boot and additional Node/Express elements shown in the poster represent the intended distributed-service direction; the repository does not contain a Spring Boot application, and the retained Node/Express mock API is a fallback/demo service. This separation allows the frontend contract to remain stable while the backend evolves from the mock Express layer to the real FastAPI and database stack.

4.2 Description of Each Layer

The **Presentation Layer** is the React + Vite single-page application, containing the login, catalog, dashboard, course detail, doubt solver and instructor screens.

The **API Layer** is built with FastAPI and Pydantic. It exposes the HTTP endpoints, validates requests, performs authentication and shapes responses.

The **Business Logic Layer** is the Python service layer, which handles course operations, enrollment and doubt orchestration, including the RAG pipeline.

The **Repository Layer** holds the database-specific queries and persistence operations, using SQLAlchemy / asynchronous SQL for PostgreSQL and Motor for MongoDB.

The **Data Layer** consists of PostgreSQL for transactional records, pgvector for vector embeddings and MongoDB for flexible assets.

The doubt-solving data flow through these layers is as follows:

1. The student selects a course, optionally a module, and submits a question.
2. POST /doubts creates a doubt row.
3. The question is transformed into a deterministic 1536-dimensional vector.
4. pgvector compares the query vector against content_embeddings using cosine distance and returns the top three chunks.
5. The answer generator checks the curated deep-knowledge base and otherwise synthesizes relevant sentences from the retrieved content.
6. A doubt response and source-reference rows are inserted.
7. The database trigger marks the doubt as answered and the transaction is committed.
8. The API returns the question, answer, confidence score and citations to the frontend.

4.3 Deployment Architecture

The submitted repository contains separate deployment configurations: Vercel for the Vite frontend, a Render configuration for the legacy Express mock API, and Dockerfiles with a Compose file for a containerized full-stack setup. The public frontend URL is Vercel-hosted. The project also documents the PostgreSQL and MongoDB environment variables, JWT configuration and local service ports.

CHAPTER 5: DESIGN

5.1 Database Design Principles

Transactional academic records are kept in normalized PostgreSQL relations, and foreign keys make every relationship between entities explicit. UNIQUE constraints prevent duplicate email addresses and duplicate student-course enrollments. Course structure is separated from individual content items so that modules can be ordered independently. AI responses and source citations are stored as separate entities to preserve provenance, and embeddings are kept in a dedicated vector table so that retrieval data can be regenerated independently. MongoDB is used for flexible media and attachment metadata rather than forcing heterogeneous attributes into relational columns.

5.2 ER Diagram / Database Schema

The relational model covers users, enrollments, courses, modules, course content, doubts, responses, sources and embeddings, as shown in Figure 5.1. The diagram also shows the two MongoDB collections (course_assets and doubt_attachments), which are treated as cross-database logical entities.

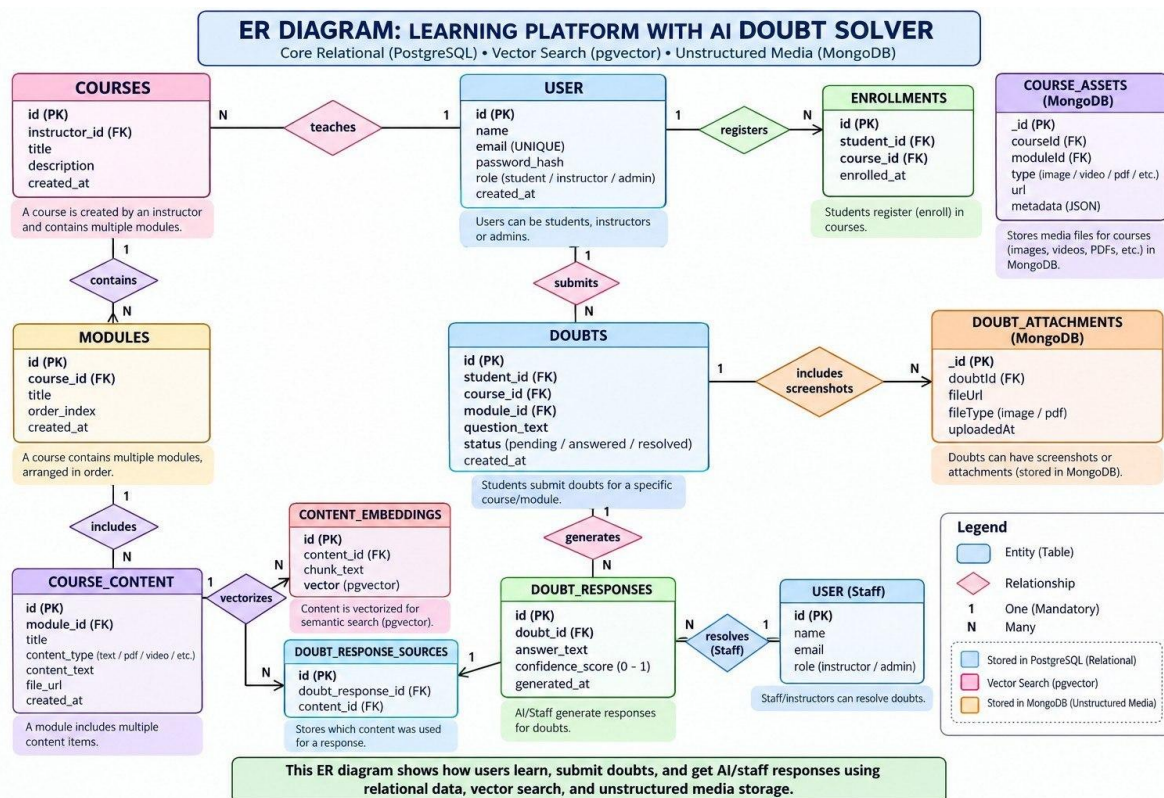


Figure 5.1: Entity-Relationship diagram of the database design

users stores students, instructors and administrators (PK id; UNIQUE email; role ENUM). **courses** holds academic course metadata (PK id; instructor FK; UNIQUE course_code). **enrollments** is the student-course association (UNIQUE on student_id and course_id). **modules** are the ordered sections inside a course, and **course_content** holds the learning items and the text used for retrieval (module FK; content type ENUM). **content_embeddings** stores the 1536-dimensional vector chunks against a content FK. **doubts** stores student academic

questions (student, course and module FKs; status ENUM), **doubt_responses** stores AI or instructor response records with a confidence score, and **doubt_response_sources** links each response to the content it cited.

MongoDB holds two collections. **course_assets** (courseId, moduleId, type, title, url, tags, uploadedBy, metadata) stores flexible metadata for videos, PDFs, slides, notes and references, and **doubt_attachments** (doubtId, fileId, fileType, uploadedAt) stores optional screenshots or files attached to student doubts.

5.3 Design Decisions and Assumptions

The following design decisions were made explicitly and are recorded here rather than left implicit in the code:

- **Cross-database references:** the identifiers linking MongoDB documents to PostgreSQL records are logical references rather than foreign keys. This keeps the relational database authoritative for academic relationships while allowing MongoDB documents to evolve independently.
- **Advanced SQL objects:** the trigger `trg_mark_doubt_answered` updates the parent doubt to answered after a response is inserted; the stored procedure `enroll_student(p_student_id, p_course_id)` inserts an enrollment only when the pair does not already exist; the view `course_doubt_analytics` aggregates total, pending, answered and escalated doubts per course; and indexes on student, course, status, content and embedding columns reduce lookup cost.
- **Normalization:** user identity, course metadata, enrollments, modules, content and responses are each stored once, which avoids update anomalies and keeps repeating groups out of master tables. The model is suitable for 3NF/BCNF-oriented reasoning, and source references are normalized through a junction table instead of duplicating content information inside every response.
- **Transaction design:** the doubt row, response row and source-link rows are committed together, so a failure before commit does not intentionally leave a half-written response or source state. The response and its citations form one logical unit.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The backend follows a layered folder architecture: `app/main.py` → `routers/` → `services/` → `repositories/` → `models/schemas` → PostgreSQL/MongoDB. This makes API concerns, business logic and database access easier to explain and test independently, with Pydantic schemas defining the API contract and SQLAlchemy ORM models representing the relational tables. It is organized into the following modules:

- **Authentication:** register and login routes, bcrypt hashing, JWT creation and token verification.
- **Users:** a user lookup endpoint and a role-aware user model.
- **Courses:** catalog listing, the detailed course tree and the instructor analytics endpoint.
- **Enrollments:** listing enrolled courses and calling the PostgreSQL stored procedure for enrollment.
- **Doubts:** create, list and retrieve doubts, with end-to-end RAG response generation.
- **RAG:** question embedding, pgvector similarity search, deep-knowledge matching and answer synthesis.

The most involved piece of logic is the retrieval-augmented doubt workflow. The function `embed_text()` produces a deterministic 1536-dimensional vector by hashing tokens and generating normalized pseudo-random projections; this is a deterministic stand-in for a production embedding model and needs no external AI API key. The similarity repository sends the vector to PostgreSQL and uses pgvector's cosine-distance operator, joining embeddings with course-content and module metadata and returning the three closest chunks. The answer-generation layer first checks the project's deep-knowledge base for a high-scoring topic match and formats a structured academic answer when one is found; otherwise it ranks the retrieved sentences and synthesizes a response from the available course material.

The current implementation should be described as a retrieval-oriented AI doubt solver rather than a cloud-hosted LLM integration. The embedding and answer-generation functions are deliberately isolated so that OpenAI, Gemini, sentence-transformers or another model can be plugged in later. Each generated response can carry source references through `doubt_response_sources`, and the frontend displays these as citation cards so that the student can see which course-content items were used as context.

6.2 Front-end Logic

The frontend uses React Router to provide protected routes: a user who is not authenticated is redirected to the login screen. After authentication, the sidebar provides access to the dashboard, catalog, doubt solver and doubt history, and the instructor route is protected by role. Course detail pages expose ordered modules and content types such as text, PDF, video and slide references. The interface uses an academic ledger metaphor — inquiry entries, resolution entries, confidence strips and source cards — which presents the system as a structured learning tool rather than an unbounded chatbot.

6.3 API Integration

The frontend API client stores the token in local storage and automatically sends Authorization: Bearer <token> with API requests through a central Axios interceptor. This keeps authentication logic centralized rather than duplicated across individual pages. The backend API surface is summarized below.

Method	Endpoint	Purpose
POST	/auth/register	Create user and return JWT
POST	/auth/login	Authenticate user and return JWT
GET	/users/{id}	Return user profile
GET	/courses	List course catalog
GET	/courses/{id}	Return course with modules/content
GET	/courses/{id}/analytics	Instructor/admin course doubt analytics
GET / POST	/enrollments	List or create enrollment
GET	/doubts?studentId=	Student doubt history
GET	/doubts?instructorId=	Instructor doubt inbox
GET	/doubts/{id}	Doubt detail with response/sources
POST	/doubts	Submit doubt and execute RAG pipeline
GET	/health	Service health check

6.4 Authentication and Authorization

Registration checks whether the email already exists, hashes the password with bcrypt and returns a JWT. Login verifies the password against the stored hash and creates a token containing the user identifier and role. Protected dependencies decode the token and load the corresponding user. The require_role() dependency factory rejects users whose role is not permitted for an endpoint, which protects the instructor/admin analytics route.

CHAPTER 7: FEATURES

7.1 List of Main Features

Student-facing features:

- Register and log in using email and password
- Dashboard showing enrolled courses, doubt counts and recent inquiries
- Search and browse the course catalog
- View course details, including modules and content items
- Enroll in a course using an idempotent backend operation
- AI Doubt Solver: select a course/module, submit a question and receive an answer with a confidence score and sources
- My Doubts: historical questions with answered / pending / escalated filters

Instructor-facing features:

- Role-protected Doubt Inbox aggregating questions raised across the instructor's courses
- Course-level doubt analytics (total, pending, answered and escalated doubts)

7.2 How Each Feature Works

AI Doubt Solver: the student selects a course and optionally a module, then submits a question. The backend stores the doubt, embeds the question, retrieves the top three relevant content chunks with pgvector, generates the answer, stores the response with its source references, and returns the answer, confidence score and citations. The frontend shows the answer together with source cards.

Enrollment: enrolling calls the `enroll_student` stored procedure, which inserts an enrollment only if the student-course pair does not already exist, so repeated requests never create duplicates.

Doubt History: the My Doubts screen acts as a persistent academic ledger. Each row includes the student's question, the course/module context, the date and the status, and filters allow the user to isolate answered, pending or escalated entries.

Instructor Doubt Inbox: the inbox is protected by role and gives the faculty member a compact review queue with counts for total inquiries, answered, pending and escalated entries, rather than exposing the entire student dashboard.

Course Analytics: the `course_doubt_analytics` SQL view summarizes doubt activity per course, and the analytics endpoint is restricted to instructor/admin roles. Total Doubts counts all doubts linked to the course; Pending counts questions without an answered state; Answered counts questions whose response insertion triggered the answered status; and Escalated counts questions explicitly marked for faculty-level review. This separates transactional tables from read-oriented reporting logic.

CHAPTER 8: TESTING

8.1 Postman and Swagger API Testing

The project poster lists Postman as the API-testing tool, while the backend repository provides pytest-based automated coverage. In addition, the FastAPI application exposes Swagger UI at /docs and ReDoc at /redoc through its standard framework configuration, providing a convenient interface for manual endpoint inspection and for trying the authentication, course, enrollment and doubt endpoints while the backend is running.

8.2 Integration Testing

Testing was organized around the highest-risk flows: authentication, course retrieval, protected analytics, enrollment and the complete doubt workflow. The repository contains 17 pytest test functions across authentication, course and doubt test modules, which use asynchronous HTTP clients and transaction-oriented database fixtures. The end-to-end doubt test verifies that the API returns an answer, a confidence score and a source list, while a separate test checks that the database trigger has changed the persisted doubt status to answered after the response insertion. This validates both application behavior and database-side behavior.

8.3 Tools Used for Testing

- pytest, pytest-asyncio and pytest-cov — automated backend tests and coverage
- httpx — asynchronous HTTP client used by the tests
- Postman — API testing, as listed in the project poster
- Swagger UI / ReDoc — manual endpoint inspection through the FastAPI /docs and /redoc pages

8.4 Test Cases and Results

The test cases below are derived directly from the assertions in the test files included in the submitted backend package, rather than from an external test plan.

#	Test Case	Expected Result	Actual Result
1	Register new user	201 and access token returned	
2	Register duplicate email	409 Conflict	
3	Login with correct password	200 and bearer token	
4	Login with wrong password	401 Unauthorized	
5	Login with unknown email	401 Unauthorized	
6	List courses	200 and list payload	
7	Get missing course	404 Not Found	
8	Course detail structure	id, title and modules present	
9	Analytics without authentication	401 Unauthorized	

#	Test Case	Expected Result	Actual Result
10	List student enrollments	200 and course list	
11	Create doubt end-to-end	201 with response and sources	
12	Doubt status after response	Status becomes answered	
13	List student doubts	200 and list payload	
14	List instructor doubts	200 and list payload	
15	Get missing doubt	404 Not Found	
16	RAG response source shape	Source entries contain contentId/title	
17	Doubt answer confidence	confidenceScore returned	

Quality observations: Pydantic schemas validate authentication, enrollment and doubt request structures; bcrypt hashing and JWT verification are implemented; a role dependency protects instructor/admin analytics; foreign keys, UNIQUE constraints, enums and SQL objects protect database integrity; doubt response and source writes are committed as one logical operation; and missing resources and invalid credentials return explicit HTTP errors. The tests provide meaningful evidence for the current backend, but they do not constitute full production-scale performance, security or load testing, which is identified as a future enhancement.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

For local development and demonstration, the application is run as follows:

- Load the database: PostgreSQL + pgvector with `db/schema.sql` and `db/seed.sql`; MongoDB with `db/mongo-schemas.js` and `db/seed-mongo.js`
- Start the backend: `cd backend`, create a virtual environment, `pip install -r requirements.txt`, then `uvicorn app.main:app --reload --port 8000` (Swagger UI at `/docs`, ReDoc at `/redoc`)
- Start the frontend: `cd frontend`, `npm install`, `npm run dev`
- Run the tests: `cd backend`, then `pytest`

Alternatively, the deployment directory contains Dockerfiles and a Compose file for the PostgreSQL/pgvector, backend and frontend services. The PostgreSQL service uses the `pgvector/pgvector:pg16` image, which provides a reproducible environment for demonstrating the polyglot architecture.

9.2 Environment Configuration

The backend uses environment variables for `DATABASE_URL`, `MONGO_URL`, `MONGO_DB_NAME`, `JWT_SECRET` and token expiry. Secrets remain outside source control, and the repository includes a `.env.example` file that documents the required configuration without embedding private credentials.

9.3 Hosting of Frontend and Backend

The frontend is deployed on Vercel at <https://dbse-dbd.vercel.app/dashboard>. The repository contains a Vercel configuration that builds the Vite application and rewrites client-side routes to `index.html`, which is required for React Router navigation. The real FastAPI backend is run locally or through the Docker Compose setup. A Render configuration is also supplied for the legacy Express mock API; this mock API remains useful as a fallback demonstration when a database-backed backend is unavailable, but it should not be confused with the real FastAPI implementation documented in Chapters 3 and 6.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Migrating from a mock Express API to a real FastAPI backend while preserving the frontend response contract.
- Maintaining consistency between PostgreSQL relational records and vector-search data.
- Designing a useful retrieval workflow without requiring an external embedding API key.
- Supporting both PostgreSQL and MongoDB connection layers in an asynchronous backend.
- Keeping multi-step doubt creation atomic so that response and source records are not partially committed.
- Protecting instructor-only analytics while keeping public catalog operations simple.

10.2 Solutions Applied

- A layered routers/services/repositories structure isolates the migration from the frontend.
- A dedicated content_embeddings table separates vector data from the core content rows.
- The deterministic embedding function is isolated and replaceable.
- Motor is initialized lazily and closed during FastAPI application shutdown.
- Doubt response and source inserts are committed together in one transaction.
- JWT and role dependencies centralize authentication and authorization.

10.3 Current Limitations or Known Bugs

- The current embedding function is deterministic and is not a semantic neural embedding model.
- The current answer generator is knowledge-base/template based rather than a live external LLM call.
- MongoDB collections are prepared, but direct asset/attachment CRUD endpoints are not exposed in the current API.
- The poster shows intended microservice concepts, but no Spring Boot service implementation is present in the submitted source.
- Performance, load testing and public backend observability are not demonstrated as a production-grade deployment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Replace deterministic embeddings with sentence-transformers, OpenAI embeddings, Gemini embeddings or another production model.
- Replace template/knowledge-base answer generation with a grounded LLM that receives retrieved course chunks as context.
- Add confidence calibration and explicit “insufficient context” responses.
- Support conversational follow-up questions while preserving course/module context.
- Add multimodal doubt attachments and image-aware question answering.
- Add instructor response workflows and escalation handling.
- Provide learning analytics, weak-topic identification and personalized recommendations.
- Add notifications for new course material and instructor responses.

11.2 Possible Integrations or Optimizations

- Split authentication, course management and doubt solving into independently deployable services.
- Introduce an API gateway and service discovery for a true microservice deployment.
- Add asynchronous queues for embedding generation and large-content ingestion.
- Introduce caching for frequently requested catalog and availability-style queries.
- Add observability using structured logs, metrics and tracing.
- Deploy PostgreSQL, MongoDB and the backend using managed cloud services.
- Provide CI/CD pipelines and automated regression tests.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The Learning Platform with AI Doubt Solver demonstrates how a modern educational application can combine a React interface, a layered FastAPI backend and multiple data models. The system organizes academic information in PostgreSQL, prepares flexible MongoDB document structures and uses pgvector to support retrieval from course-content embeddings.

12.2 What Was Achieved

- A functional React learning-platform interface with protected navigation.
- A real FastAPI backend organized into routers, services and repositories.
- JWT authentication and bcrypt password hashing.
- Course catalog, course details and enrollment workflows.
- A persistent doubt ledger for students and an instructor-facing doubt inbox.
- A retrieval-oriented doubt solver with vector similarity search and source references.
- Advanced PostgreSQL objects: a trigger, a stored procedure and an analytics view.
- MongoDB schemas and asynchronous database connection support.
- Automated pytest tests covering 17 backend test functions.
- Vercel, Docker and Render deployment configurations for different demonstration scenarios.

12.3 Skills Learned

The project provided practical experience in database normalization, foreign-key design, advanced SQL, stored procedures, triggers, views, vector databases, NoSQL modeling, REST API design, asynchronous Python, JWT security, React routing, automated testing and deployment. It also provided experience in designing an AI-oriented retrieval pipeline whose individual stages can be replaced independently as the system evolves.

Overall, the submitted system establishes a strong foundation for a production learning platform while clearly separating the current academic implementation from the future enhancements required for real-world AI and distributed deployment.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev/>
- Vite — Official Documentation, <https://vite.dev/>
- FastAPI — Official Documentation, <https://fastapi.tiangolo.com/>
- SQLAlchemy — Documentation, <https://www.sqlalchemy.org/>
- PostgreSQL — Documentation, <https://www.postgresql.org/docs/>
- pgvector — Project Documentation, <https://github.com/pgvector/pgvector>
- MongoDB — Documentation, <https://www.mongodb.com/docs/>
- Motor — Documentation, <https://motor.readthedocs.io/>
- Pydantic — Documentation, <https://docs.pydantic.dev/>
- Python-JOSE / JWT ecosystem — Documentation, <https://python-jose.readthedocs.io/>
- pytest — Documentation, <https://docs.pytest.org/>
- Axios — Documentation, <https://axios-http.com/>
- Docker — Documentation, <https://docs.docker.com/>
- Vercel — Documentation, <https://vercel.com/docs>
- Postman Learning Center, <https://learning.postman.com/>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org/>
- Submitted project source: DBSE project backend, frontend, db and deployment directories.
- Submitted project poster: Learning Platform with AI Doubt Solver.

Note: the implementation claims in this report are based on the source files supplied with the project. Where the project poster describes a broader microservice architecture than the current source code, this report distinguishes the implemented FastAPI backend from the proposed/illustrative distributed-service components.

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

Representative SQL used in the project:

```
-- Trigger: mark a doubt answered after response insertion
CREATE TRIGGER trg_mark_doubt_answered
AFTER INSERT ON doubt_responses
FOR EACH ROW EXECUTE FUNCTION mark_doubt_answered();

-- Idempotent enrollment procedure
CALL enroll_student(:student_id, :course_id);

-- pgvector retrieval concept
ORDER BY ce.embedding <=> CAST(:vec AS vector)
LIMIT 3;
```

Appendix B: Installation and Setup Instructions

- Backend: `cd backend && python -m venv .venv && pip install -r requirements.txt && uvicorn app.main:app --reload --port 8000`
- Frontend: `cd frontend && npm install && npm run dev`
- PostgreSQL + pgvector: load `db/schema.sql` and `db/seed.sql`
- MongoDB: load `db/mongo-schemas.js` and `db/seed-mongo.js`
- Tests: `cd backend && pytest`

Appendix C: Default Login Credentials

The login page of the application (LearnAI) shows the default password as “password123” and provides direct-access “Student demo” and “Faculty demo” buttons. The demonstration accounts seen in the screenshots in Appendix E are Ravi Kumar (Student) and Dr. Ananya Sharma (Instructor). The seeded accounts are defined in `db/seed.sql`.

Appendix D: API Reference Summary

The backend exposes the endpoints listed in Section 6.3 (authentication, users, courses, enrollments, doubts and health). Interactive request/response documentation is available through Swagger UI (`/docs`) and ReDoc (`/redoc`) when the backend is running.

Appendix E: Screenshots

The following screenshots were taken from the running LearnAI application (React frontend with the FastAPI backend), and the project poster is shown as Figure 14.1.

1. Login Page

LearnAI

Course Catalog & Academic Doubt Ledger

Sign In

Register

Institutional email

Password (default: password123)

ROLE DESIGNATION

Student

Browse catalog & submit inquiries

Faculty

Curriculum oversight & doubt review

Processing...

DIRECT DEMONSTRATION ACCESS

Student demo

Faculty demo

Database Systems & Engineering Project

PostgreSQL · pgvector RAG · FastAPI Architecture

2. Student Dashboard

LearnAI

CATALOG & LEDGER

INDEX

Dashboard

Course Catalog

Doubt Solver

My Doubts

SESSION

Sign out

Welcome back, Ravi

Current status of course enrollments and academic inquiry ledger.

Ask doubt

4

ENROLLED COURSES

Active curriculum

3

TOTAL DOUBTS

Questions logged

2

ANSWERED

Resolved by system

0

PENDING

Awaiting resolution

Enrolled Courses

View catalog

Recent Inquiries

View ledger

Machine Learning

Instructor: LakshmiPrasanna Yeluri — 6 modules

[25SC2107E]

Data Structures and Algorithms - 3

Instructor: Dr. B. Tirumala Rao — 6 modules

[25CS2103E]

Database Systems Engineering and Distributed Backend Development

Instructor: Dr. V. S. Rao — 6 modules

[25CS1302E]

Operating Systems and Systems Programming

Instructor: Dr. K. Venkat — 6 modules

[25CS2104E]

What is the difference between 2NF and BCNF? My notes say BCNF is stricter but I don't understand why a relation in 3NF might not be in BCNF.

Machine Learning / M2, Supervised Learning — Linear Models at Depth

✓ Answered

When should I use a Hash index over a B-Tree index in PostgreSQL?

Machine Learning / M3, Supervised Learning — Tree-Based Models

✓ Answered

I understand CAP theorem says you can only pick 2 of 3, but in practice most systems seem to always be partition-tolerant. Does that mean we're really just choosing between CP and AP?

Data Structures and Algorithms - 3 / Module 1, TextHack as a System — The Advanced-Algorithm Question Bank

■ Escalated

Page 27

3. Course Catalog

LearnAI

CATALOG & LEDGER

INDEX

Dashboard

Course Catalog

Doubt Solver

My Doubts

SESSION

Sign out

Course Catalog

Academic curriculum directory and syllabus index.

Filter courses by title or subject...

[25SC2107E]

Machine Learning
Machine Learning covers statistical and algorithmic learning systems, model evaluation, and deployment engineering anchored to live prediction services. Covers the classical machine learning canon including...
Coordinator: LakshmiPasanna Yeluri • 4 Credits • Prereq: 25SC1306E - Computational Foundations for Artificial Intelligence

6 modules

View course

[25SC2103E]

Data Structures and Algorithms - 3
Advanced algorithmic design and analysis covering advanced string algorithms, multi-dimensional dynamic programming, network flow, NP-completeness reductions, approximation algorithms, and randomised data...
Coordinator: Dr. B. Tirumala Rao • 4 Credits • Prereq: 25SC1301E - Data Structures and Algorithms - 2

6 modules

View course

[25SC1302E]

Database Systems Engineering and Distributed Backend Development
Enterprise database engineering and distributed backend architectures covering relational theory, advanced SQL, polyglot persistence (SQL, NoSQL, Vector), transaction processing, MVCC, distributed consensus, query...
Coordinator: Dr. V. S. Rao • 4 Credits • Prereq: None (Core Course)

6 modules

View course

[25SC2104E]

Operating Systems and Systems Programming
Operating system abstractions and low-level systems programming in C on Linux. Covers CPU privilege modes,

6 modules

View course

4. AI Doubt Solver

LearnAI

CATALOG & LEDGER

INDEX

Dashboard

Course Catalog

Doubt Solver

My Doubts

SESSION

Sign out

Doubt Solver

Academic inquiry desk referencing course syllabus documents and textbook excerpts.

Select course catalog entry...
Select course catalog entry...
Machine Learning
Data Structures and Algorithms - 3
Database Systems Engineering and Distributed Backend Development
Operating Systems and Systems Programming

Inquiry Ledger Ready
Select a course catalog entry above and record your question. The system will retrieve matching syllabus records and provide verified answers.

Select a course above to begin...

Submit doubt

Vector retrieval engine references PostgreSQL pgvector embeddings and course text chunks.

5. My Doubts (Doubt History)

LearnAI

CATALOG & LEDGER

INDEX

Dashboard

Course Catalog

Doubt Solver

My Doubts

SESSION

Sign out

My Doubts

Historical ledger of submitted inquiries and resolutions.

Ask doubt

All (3) Answered (2) Pending (0) Escalated (1)

What is the difference between 2NF and BCNF? My notes say BCNF is stricter but I don't understand why a relation in 3NF might not be in BCNF.

Machine Learning / M2. Supervised Learning — Linear Models at Depth / Logged: 20 Aug 2025

✓ Answered
Answer on file

When should I use a Hash index over a B-Tree index in PostgreSQL?

Machine Learning / M3. Supervised Learning — Tree-Based Models / Logged: 22 Aug 2025

✓ Answered
Answer on file

I understand CAP theorem says you can only pick 2 of 3, but in practice most systems seem to always be partition-tolerant. Does that mean we're really just choosing between CP and AP?

Data Structures and Algorithms - 3 / Module 1. TextHack as a System — The Advanced-Algorithm Question Bank / Logged: 03 Sept 2025

■ Escalated
Answer on file

6. Instructor Dashboard

LearnAI
CATALOG & LEDGER

INDEX

Dashboard

Course Catalog

Doubt Inbox

SESSION

Sign out

D Dr. Ananya Sharma
Instructor

Welcome back, Dr.

Current status of course enrollments and academic inquiry ledger.

Ask doubt

0

ENROLLED COURSES
Active curriculum

0

TOTAL DOUBTS
Questions logged

0

ANSWERED
Resolved by system

0

PENDING
Awaiting resolution

Enrolled Courses

View catalog

Recent Inquiries

View ledger

No active enrollments recorded.

No inquiries recorded in ledger yet.

Academic Doubt Resolution Desk

Submit questions regarding any enrolled module. The system references textbook materials and course documents to provide verified citations.

Ask doubt

Project Poster / Visual Evidence

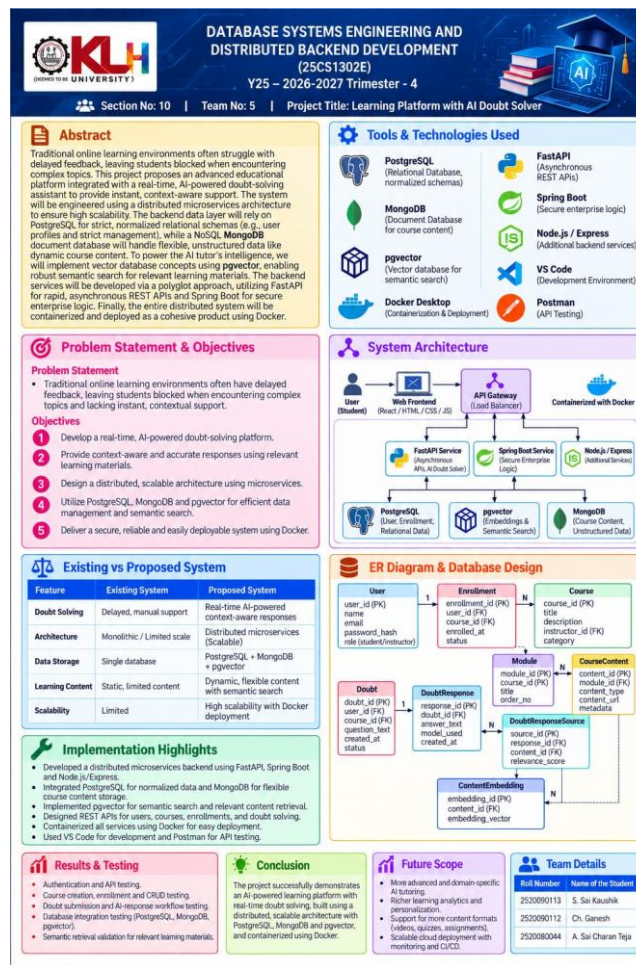


Figure 14.1: Supplied project poster. Demonstration URL: <https://dbse-dbd.vercel.app/dashboard>